

Technische internetbasierte Systeme

Smart Home

Berkan Demir (78096),
Benjamin Leonhardt (3018332)

Demo

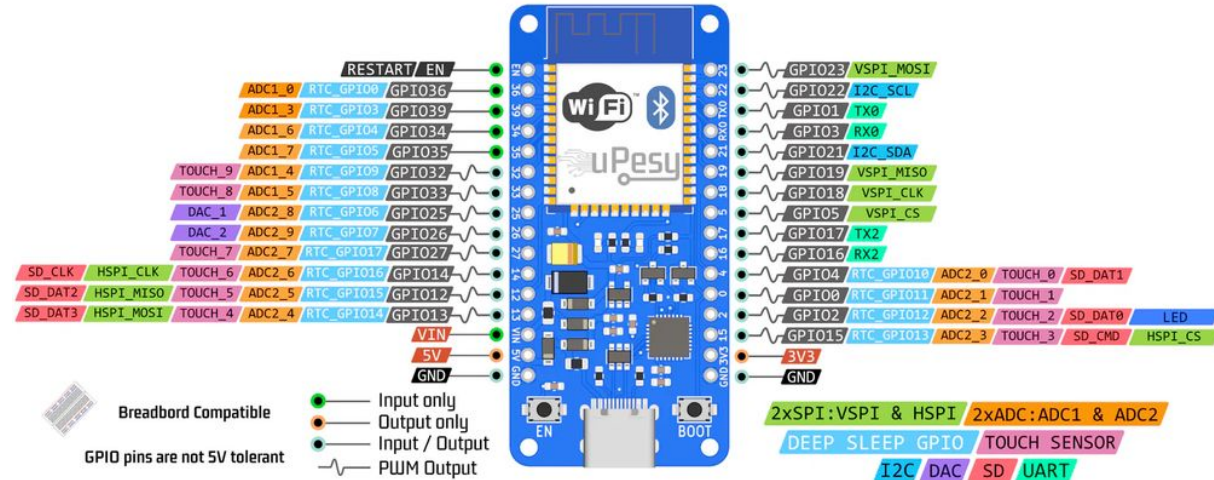
Use Case und Projektidee

- Automatisierung von Alltagsaufgaben (Ventilator und Jalousie steuern)
- Darstellung von aktuellen Daten auf einer Webseite, sowie auf einem E-Ink Display
- Wetter vorhersage
- Wecker/Alarm zu einer bestimmten Uhrzeit

Hardware: yPesy ESP32 Wroom

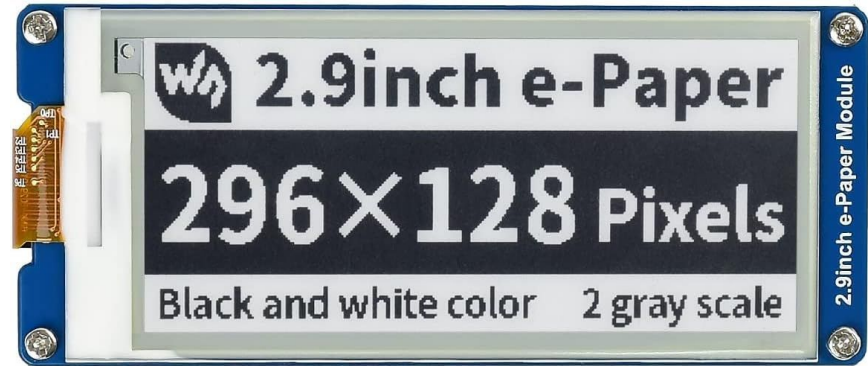
- Dual Core
- 240 Mhz
- 4MB Flash
- 40 Pin
- WiFi & BT

ESP32 Wroom DevKit Rev2 Full Pinout



Hardware: E-Ink Display

- 2.9"
- 296x128 Pixel
- Schwarz / Weiß
- SPI Anschluss



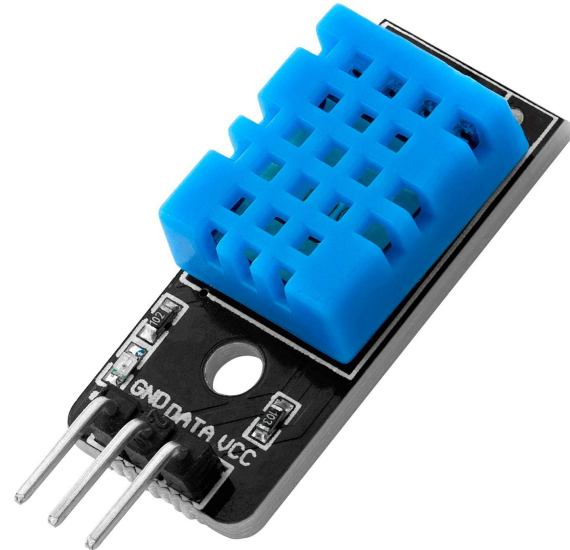
FRONT



BACK

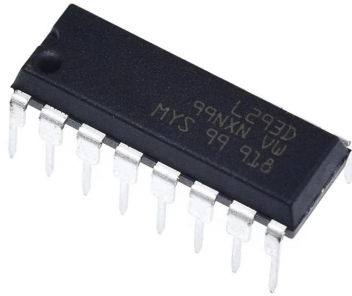
Hardware: DHT11

- Luftfeuchtigkeits- und Temperatursensor
- 0-50 C°
- 20-90 % Luftfeuchtigkeit



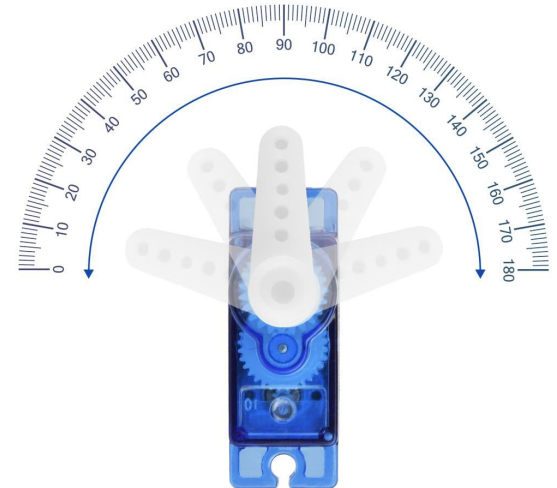
Hardware: DC-Motor und Treiber L293D

- Gleichstrom Elektromotor
- 3-5v
- Treiber steuert richtung in die sich der Motor dreht
- Motor simuliert Ventilator



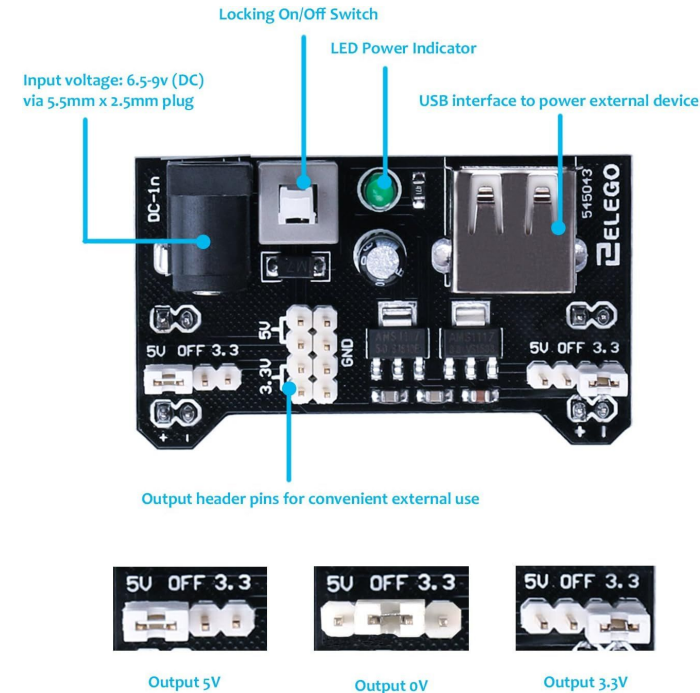
Hardware: Photodiode und Servomotor

- Servo: SG90
- 0,1 Sekunden pro 60 Grad
- PWM steuerung
- Diode steuert den Servomotor
- Je Helligkeit wird auf oder zugemacht
- Simuliert die Jalousie



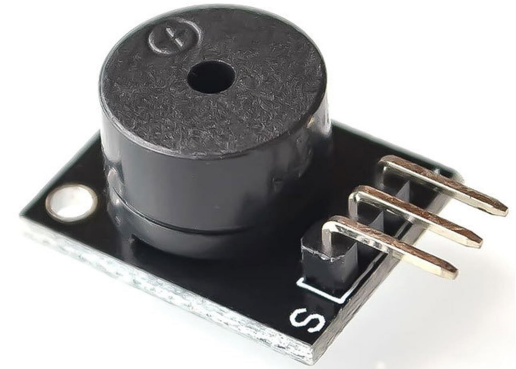
Hardware: Power supply für Breadboard

- Jumperbar für 3,3 und 5 Volt
- Für bessere stabilität bei vielen Sensoren / Aktoren
- Wird auf Breadboard gesteckt
- Strom wird mit DC Netzteil gespeist



Hardware: Buzzer und Button

- Buzzer gibt ton vom Wecker/Alarm aus
- Mit Button kann der Alarm ausgeschaltet werden



Schaltplan

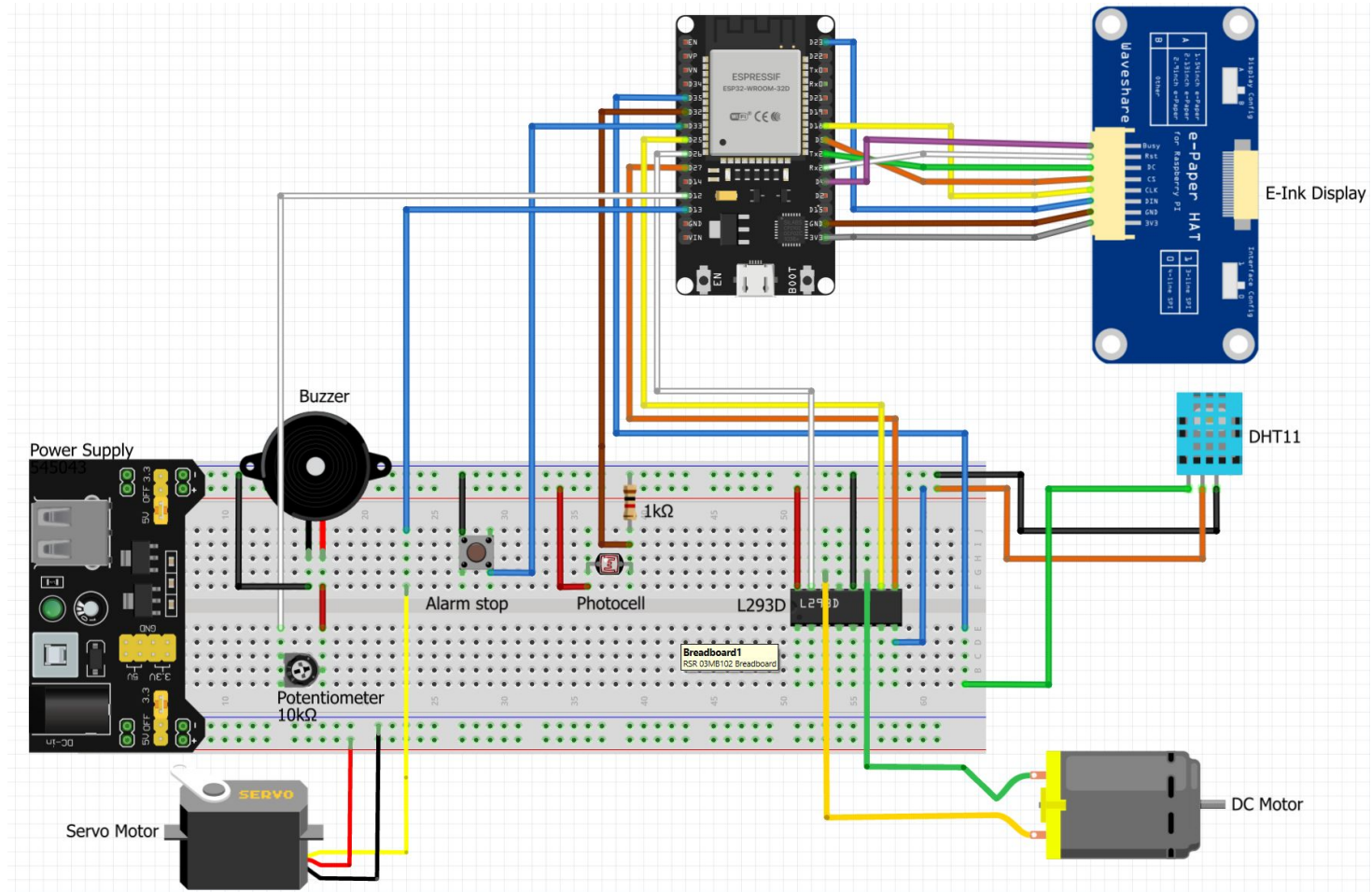
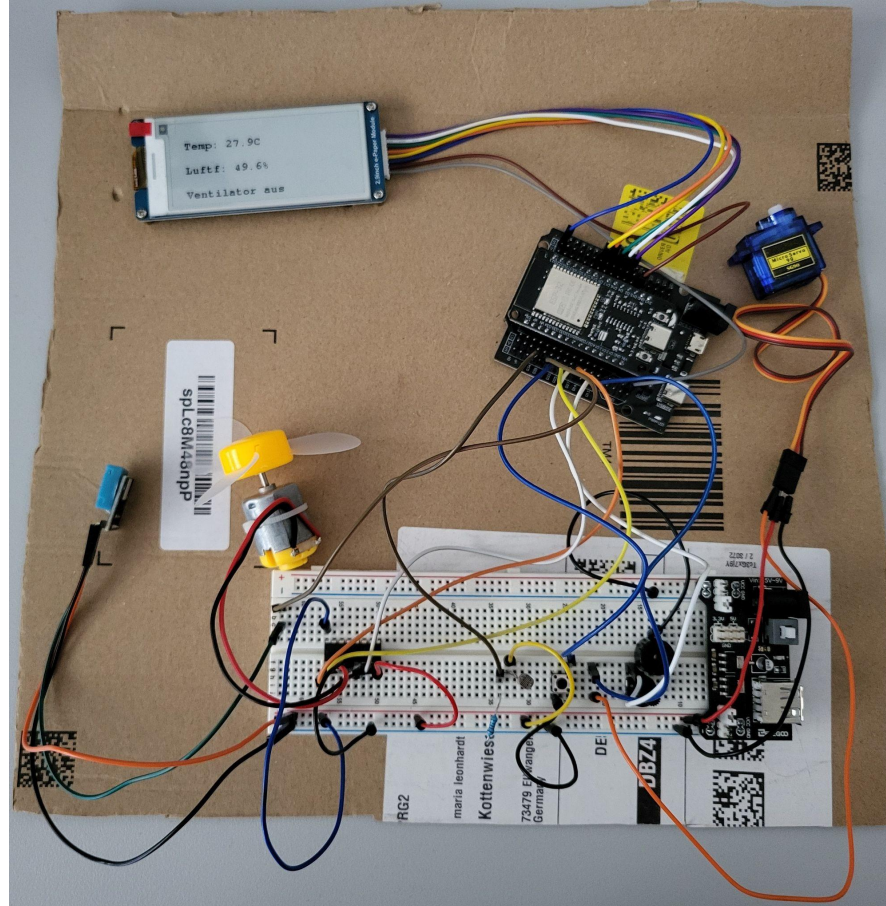


Bild vom Projekt



Software: Multithreading

- Einzelne gekapselte Threads

```
804 // Start threads
805 xTaskCreatePinnedToCore(webServerTask, "WebTask", 8192, NULL, 3, &webHandle, 0);
806 xTaskCreatePinnedToCore(readWeatherForecast, "WeatherTask", 16384, NULL, 1, &forecastHandle, 1);
807 xTaskCreatePinnedToCore(updateDisplay, "DisplayTask", 16384, NULL, 2, &displayHandle, 1);
808 xTaskCreatePinnedToCore(readSensor, "SensorTask", 16384, NULL, 1, &readSensorsHandle, 1);
809 xTaskCreatePinnedToCore(controlFan, "FanTask", 16384, NULL, 1, &fanHandle, 1);
810 xTaskCreatePinnedToCore(controlServo, "ServoTask", 16384, NULL, 1, &servoHandle, 1);
811 xTaskCreatePinnedToCore(handleAlarm, "AlarmTask", 16384, NULL, 1, &alarmHandle, 1);
```


Software: Multithreading

- Einzelne gekapselte Threads

```
349 // ===== FAN CONTROL =====
350 void controlFan(void *p)
351 {
352     while (true)
353     {
354         if (print & 0x02)
355         {
356             Serial.println("Checking temp and setting fan status");
357         }
358
359         if ((temperature > 30.0 && !ventilatorControlWebsite) || (ventilatorOn && ventilatorControlWebsite))
360         {
361             digitalWrite(ENABLE, HIGH);
362             digitalWrite(DIRA, HIGH);
363             digitalWrite(DIRB, LOW);
364             fanOn = true;
365         }
366         else
367         {
368             digitalWrite(ENABLE, LOW);
369             fanOn = false;
370         }
371         if (print & 0x02)
372         {
373             Serial.print("Fan is ");
374             Serial.println(fanOn);
375         }
376         vTaskDelay(pdMS_TO_TICKS(FAN_DELAY));
377     }
378 }
```

Software: Display

- Loop zeichnet pro durchlauf eine Seite

```
do {  
  if (page==0) {  
    display.fillScreen(GxEPD_WHITE);  
    display.setCursor(20,40); display.print("Temp: "); display.print(temperature,1); display.print("C");  
    display.setCursor(20,80); display.print("Luftf: "); display.print(humidity,1); display.print("%");  
    display.setCursor(20,120); display.print(fanOn?"Ventilator an":"Ventilator aus");  
  } else if (page==1) {  
    display.fillScreen(GxEPD_WHITE);  
    display.setCursor(20,40); display.println("Heute ist der:");  
    display.setCursor(20, 80);display.println(datumBuffer);  
    display.setCursor(20,120);display.println(&timeinfo,"%H:%M:%S Uhr");  
  }  
}
```

Software: Alarm

- Loop gibt unterschiedliche Töne aus
- Abfrage, ob Taster gedrückt wird

```
608     if (alarmActive && timeinfo.tm_hour == weckStunde && timeinfo.tm_min == weckMinute && !alarmOff)
609     {
610         if (print & 0x20)
611         {
612             Serial.println("Alarm is buzzing and opening jalousie...");
613         }
614         servoControlWebsite = true;
615         myservo.write(0);
616         for (int n = 0; n < 8 && !alarmOff; n++)
617         {
618             alarmOff = (digitalRead(ALARM_OFF_BUTTON) == 0);
619             tone(SPEAKERPIN, melody[n], noteDuration);
620             vTaskDelay(pdMS_TO_TICKS(500));
621         }
622     }
```

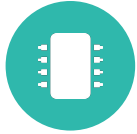
```
50 // ===== SOUND =====
51 int melody[] = {NOTE_C5, NOTE_D5, NOTE_E5, NOTE_F5,
52                | | | | | NOTE_G5, NOTE_A5, NOTE_B5, NOTE_C6};
53 int noteDuration = 500;
54 #define SPEAKERPIN 12
```

```
1  /*****
2   * Public Constants
3   *****/
4
5  #define NOTE_B0  31
6  #define NOTE_C1  33
7  #define NOTE_CS1 35
8  #define NOTE_D1  37
9  #define NOTE_DS1 39
10 #define NOTE_E1  41
11 #define NOTE_F1  44
```


Software: Photodiode und Servo

- 3x Photodiode ausgelesen und zusammen gerechnet
- “Jalousie” öffnet nur, wenn noch nicht offen

```
380 // ===== Servo CONTROL =====
381 void controlServo(void *p)
382 {
383     while (true)
384     {
385         int v = analogRead(LDRPIN); vTaskDelay(pdMS_TO_TICKS(SERVO_DELAY));
386         v += analogRead(LDRPIN); vTaskDelay(pdMS_TO_TICKS(SERVO_DELAY));
387         v += analogRead(LDRPIN); |drValue = v;
388         int s = myservo.read();
389         if(!servoControlWebsite){
390             if (v < 1200 && s < 175)
391                 myservo.write(180);
392             else if (v > 1200 && v < 2000 && (s < 85 || s > 95))
393                 myservo.write(90);
394             else if (v > 2000 && s > 5)
395                 myservo.write(0);
396             vTaskDelay(pdMS_TO_TICKS(SERVO_DELAY));
397         }
398     }
399 }
```



Smart Home – Dashboard

Web-basiertes Steuerungs- und Anzeige-Interface für ein ESP32-gestütztes Smart-Home-System

Wo das Dashboard ins System passt

Das Dashboard ist die Schnittstelle zwischen Nutzer und Hardware



Kein externer Server nötig: Der ESP32 hostet die Website selbst über WebServer.h. Jedes Gerät im selben WLAN kann das Dashboard direkt über die IP-Adresse des ESP32 aufrufen.

Ziele des Dashboards

Welche Anforderungen sollte die Oberfläche erfüllen?



Live-Daten anzeigen

Temperatur, Luftfeuchtigkeit, Jalousie-Status und Ventilator-Zustand in Echtzeit.



Fernsteuerung

Jalousie manuell setzen und Wecker direkt im Browser einstellen.



Geräteunabhängig

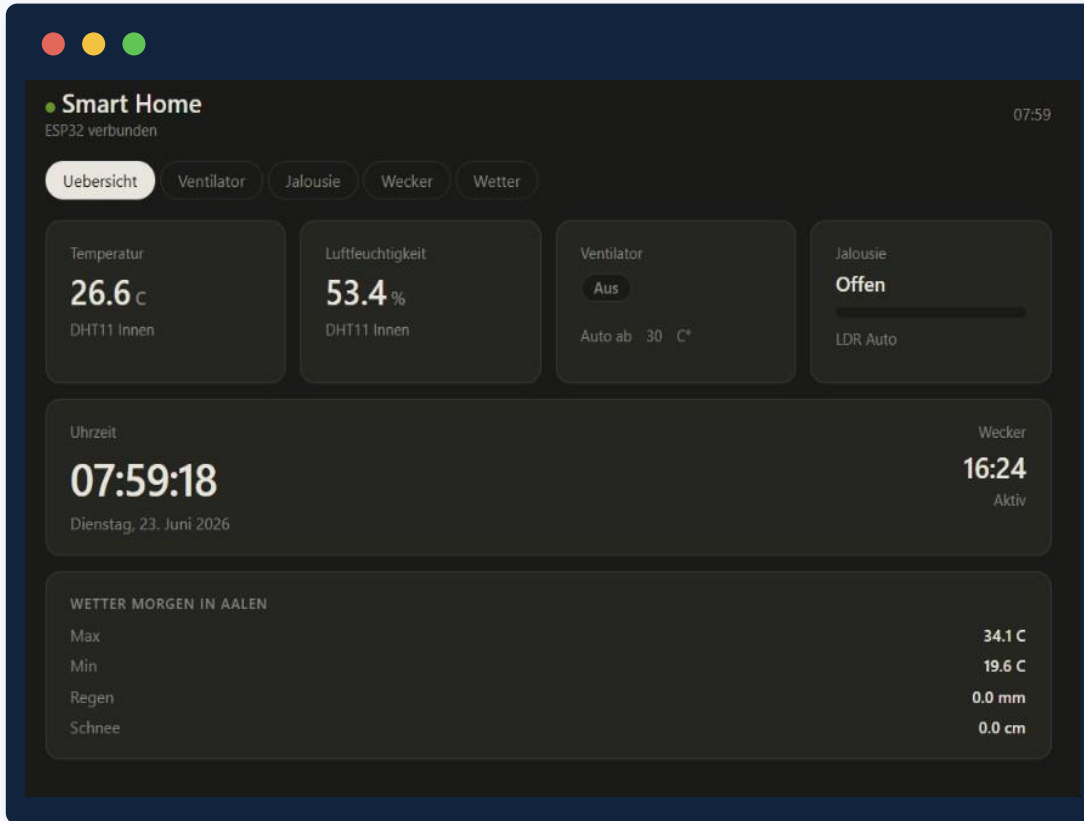
Funktioniert auf jedem Smartphone, Tablet oder PC im selben Netzwerk.



Keine Zusatz-Hardware

Läuft direkt auf dem ESP32 — kein separater Server, keine Cloud.

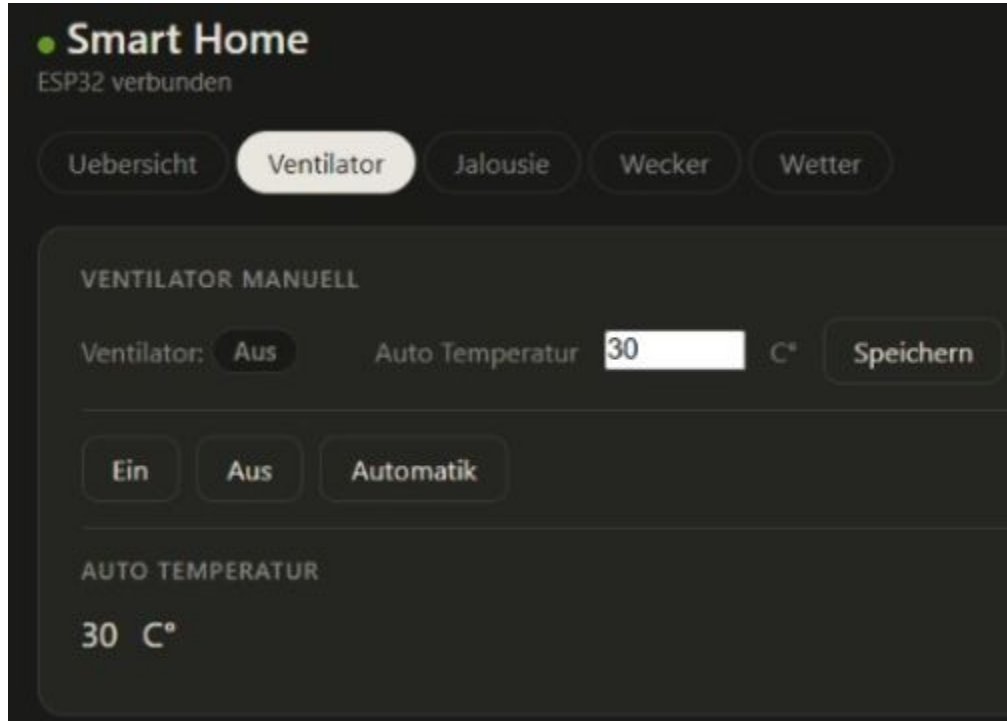
Tab „Übersicht“ — alle Werte auf einen Blick



Was hier passiert

- Vier Karten zeigen Temperatur, Luftfeuchtigkeit, Ventilator- und Jalousie-Status.
- Eine Live-Uhr läuft im Browser; der Wecker wird vom ESP32 übernommen.
- Alle 3 Sekunden fragt das Dashboard `/api/status` per Fetch erneut ab.
- Wetterdaten von Open-Meteo werden täglich vom ESP32 aktualisiert.

Tab „Ventilator“ — manuelle Steuerung



Funktionsweise



Festlegen der Auto Temperatur

Frei festlegbare Temperatur, wann der Ventilator automatisch anspringen soll



Manuell ein- und ausschalten



Marker für Status

Marker der anzeigt ob der Ventilator läuft

Tab „Jalousie“ — manuelle Steuerung



Funktionsweise



Drei feste Positionen

Buttons rufen `/api/jalousie?angle=0|90|180` auf und steuern den Servo direkt.



Live-Rückmeldung

Fensteransicht und Winkel-Anzeige aktualisieren sich automatisch beim nächsten Status-Poll.



Automatik im Hintergrund

Der LDR-Sensor passt die Jalousie auch ohne Eingabe an die Helligkeit an.

Tab „Wecker“ & „Wetter“



Der ESP32 übernimmt Stunde, Minute und Aktiv-Status direkt aus der Browser-Eingabe — kein Knopfdruck am Gerät nötig.



Datenquelle: Open-Meteo API — vom ESP32 einmal täglich abgerufen und über /api/status an das Dashboard weitergegeben.

Technische Umsetzung

Wie Frontend und ESP32-Firmware zusammenspielen



HTML / CSS / JavaScript

Dashboard liegt als String (PROGMEM) im ESP32-Code und wird komplett vom Gerät ausgeliefert.



WebServer.h

Eingebaute ESP32-Bibliothek; verarbeitet HTTP-Requests ohne Zusatz-Hardware.



ArduinoJson

Serialisiert Sensordaten als JSON für die Antwort auf /api/status.



FreeRTOS-Tasks

Webserver läuft als eigener Task parallel zu Sensor-, Servo- und Alarm-Logik.

API-ENDPUNKTE AUF DEM ESP32

GET /

Liefert das komplette Dashboard-HTML

GET /api/status

Aktuelle Sensordaten als JSON

GET /api/jalousie

Setzt den Servo-Winkel (?angle=)

GET /api/alarm

Setzt Weckzeit & Aktiv-Status

Datenfluss: Vom Sensor zum Browser



Ergebnis: Ein Live-Dashboard, das ohne externen Server, ohne App-Installation und ohne Internetverbindung im lokalen WLAN läuft.

Fazit

Was erreicht wurde

- Voll funktionsfähiges Dashboard direkt auf dem ESP32 — kein externer Server nötig
- Fünf klar getrennte Ansichten: Übersicht, Ventilator, Jalousie, Wecker, Wetter
- Steuerung und Anzeige in Echtzeit über eine einfache REST-API

Danke für die Aufmerksamkeit! Fragen?